

DELHI AIRCAST — PS-13 AI-POWERED AIR QUALITY FORECASTING

FULL TECHNICAL PROJECT REPORT

Prepared: 9 October 2026

Repository: <https://github.com/priyanshuchawda/Delhi-AirCast-PS13>

Local project path: `/root/Delhi-AirCast-PS13`

REPORT SCOPE AND EVIDENCE STANDARD

This report is based on inspection of the checked-out project documentation, Python and TypeScript source, model evaluation JSON tracked in Git, experiment ledger, deployment files, figures, and existing report deliverables. The large research data and most historical forecast artifacts are deliberately not committed to Git. As a result, reported research scores below are transcribed from the project experiment ledger or evaluation JSON; they were not retrained or independently reproduced during this report preparation. The two-site WAQI pilot weights and evaluation summary are tracked in the checkout. External demo URLs are documented in README but were not live-checked for uptime.

CONTENTS

1. Project overview and objective
2. Delivered scope and product distinctions
3. Technical stack
4. System architecture and data flow
5. Data sources, provenance, and roles
6. Acquisition and normalization implementation
7. Quality control and dataset construction
8. Feature engineering and leakage protection
9. Model program: baselines, promoted candidates, challengers
10. Training protocols and model selection
11. Results and what they mean
12. AQI computation and forecast semantics
13. API, dashboards, and user-facing behavior
14. Packaging, hosting, and operations
15. Repository map and key scripts
16. Reproduction guide
17. Strengths, limitations, risks, and recommended next work
18. Final assessment

1. PROJECT OVERVIEW AND OBJECTIVE

Delhi AirCast is an academic implementation of PS-13, “AI-Powered Air Quality Forecasting.” The motivating problem is that Delhi residents

The research forecasting task is primarily to predict CPCB/OpenCity PM2.5 concentration in micrograms per cubic metre ($\mu\text{g}/\text{m}^3$) at Delhi monitoring stations for 1, 3, 6, 12, and 24 hours ahead. In parallel, the project built a separate experimental live pilot for two monitoring areas using World Air Quality Index (WAQI) station data. These two tracks use different input populations, targets, scales, models, and service paths and must not be conflated.

The project deliberately uses a model ladder. It starts with persistence and simple tree models, then tests boosted trees, sequence networks, graph neural networks, extra data features, and an AQI composition experiment. More complex algorithms are challengers; they are selected only if evaluations show a useful gain.

2. DELIVERED SCOPE AND PRODUCT DISTINCTIONS

A. Historical CPCB research pipeline

- 39 station files covering 2024–2025 in the reported panel.
- Hourly, station-level measurements and derived features.
- Direct predictions for +1, +3, +6, +12, and +24 hours.
- Selected practical model: XGBoost per horizon, trained on the unified station/weather/CAMS/FIRMS feature table for final artifacts.
- Offline API/dashboard modes require prebuilt historical feature and model bundles; they do not query live CPCB for the national station

B. AQI estimates

- Official CPCB breakpoint/sub-index logic is implemented for PM2.5, PM10, NO2, SO2, O3, CO, and NH3.
- If too few pollutants are supplied, the implementation returns an explicit insufficiency state.
- If only PM2.5 is available, the output is explicitly a PM2.5-derived proxy.
- An experimental six-hour five-pollutant forecast combines PM2.5, PM10, NO2, CO, and O3. SO2 and NH3 are not forecast by that experiment.

C. Live WAQI pilot

- Two sites: Pusa (site_107) and R.K. Puram (site_124).
- Retrieves current WAQI individual PM2.5 index values server-side.

- Predicts the WAQI / US-EPA-scale individual PM2.5 index proxy, not CPCB AQI and not concentration in $\mu\text{g}/\text{m}^3$.
- Supports 1/3/6/12/24h model horizons, observation freshness checks, caching, optional history collection, and optional point-in-time
- Requires a WAQI API key and fresh upstream data. When its history is not warm, it uses a single-observation fallback.

D. Product boundary

The station map displays sensor locations and their station-level model outputs. It does not show a validated, continuously interpolated neighbourhood pollution field. Forecast artifacts labelled historical can describe past target times if the offline bundle is old. The live pilot is experimental and is not a health advisory.

3. TECHNICAL STACK

Research/data layer:

- Python 3.13+ as declared by pyproject.toml.
- pandas and NumPy for tabular/time-series manipulation and numerical features.
- PyArrow/Parquet for compressed columnar datasets.
- JSON, CSV, and source-specific manifests for raw acquisition and provenance.
- scikit-learn for HistGradientBoosting, metrics, preprocessing support, and utilities.
- XGBoost (XGBRegressor, histogram tree method) for the main multi-station tree models.
- PyTorch for LSTM, temporal convolutional network (TCN), and spatial GNN experiments.
- joblib for serialized scikit-learn/XGBoost model bundles.
- pytest for unit tests.

Backend/API:

- FastAPI with Pydantic request validation.
- Uvicorn ASGI server.
- joblib model loading; pandas/NumPy inference feature construction.
- Python urllib for WAQI and Open-Meteo live HTTP calls in the API module.
- CORS configuration through DELHI_AIRCAST_ALLOWED_ORIGINS.

Research dashboard:

- Streamlit, Plotly/pydeck-related visualization dependencies, pandas, PyTorch, and model loaders.
- Reads the unified local research table and local model artifacts; it does not fetch live readings.

Web dashboard:

- Next.js 16, React 19, TypeScript 5, SWR data fetching, Recharts visualizations, and Lucide icons.
- CSS custom UI and SVG station-map drawing; frontend rewrites /backend/* requests to the API URL.
- Node.js 22 runtime in the frontend Dockerfile.

Packaging and delivery:

- uv for Python environment and locked dependency workflow.
- npm ci/package-lock for the frontend.
- Docker and Docker Compose for API + Next.js containers.
- GitHub Actions CI runs uv sync --locked and pytest on pushes/PRs.
- Git LFS is not used for large research artifacts; raw data and most historical artifacts are ignored/separately distributed.

4. SYSTEM ARCHITECTURE AND DATA FLOW

The project can be understood as two related pipelines:

Historical research path:

Public/raw sources -> source-specific download scripts -> raw files + manifests/checksums -> normalization to hourly station/source tables -> quality audits -> point-in-time multi-station panel -> auxiliary as-of joins -> train/validation/backtest/evaluation -> model bundles + compact serving snapshot -> FastAPI or Streamlit/Next.js dashboard.

Live pilot path:

WAQI API key on server -> WAQI station feed -> timestamp/value validation and 5-minute cache -> append-only deduplicated observation history -> current/lag/spatial features -> horizon-specific HistGradientBoosting model -> optional eligible weather model if fresh issue-time weather exists -> API JSON -> web pilot panel.

Important boundaries:

- The historical CPCB concentration model does not consume WAQI index inputs.
- The WAQI model is not merely the historical CPCB XGBoost model with a new endpoint; it is a separate index-scale model with a separate training pipeline.
- The API serves historical concentration models from files in a data bundle and serves live WAQI index values through external calls.
- The frontend proxies API calls via Next.js rewrites; the WAQI token is only used server-side.

5. DATA SOURCES, PROVENANCE, AND ROLES

Primary supervised label source:

1. OpenCity Delhi Hourly Air Quality Reports (CPCB/DPCC historical archive). The project documents 78 downloadable resources; the normal

Official/current reference:

2. CPCB real-time AQI resource on data.gov.in. A paginated connector can collect a current snapshot and metadata with a key. It is used

Weather and regional chemistry:

3. Open-Meteo Historical Weather API: central Delhi hourly temperature, relative humidity, precipitation, wind speed/direction, pressure
4. Open-Meteo Air Quality / CAMS Global: regional PM and gas reanalysis/forecast fields. It is an auxiliary atmospheric model product,

Spatial/event/validation sources:

5. IIT Delhi AirDelhi mobile sensor and gridded data: spatial benchmark and research layer. Kept separate from CPCB reference labels because of its unique provenance.
6. NASA FIRMS MODIS/VIIRS fire hotspots: event/source proxy for Delhi and surrounding upwind regions. It is not direct ground-level panel data.
7. OpenAQ v3: catalog/latest/hourly data for cross-checks and possible validation. It does not overlap the main 2024-2025 CPCB panel source.
8. CPCB legacy 2017-2023 AQI-only archive: separate historical AQI validation layer; no pollutant-concentration labels are inferred from AQI values.
9. Kaggle and Hugging Face community/mirror data: schema and comparison sources pending individual provenance/license audits; not siloed.
10. WAQI feed: current live pilot input for the two WAQI UIDs mapped to Pusa and R.K. Puram.

Source policy is explicit: preserve source identity, units, timestamps, manifests, and hashes; do not assume similarly named pollutant fields are interchangeable; keep keys outside Git; and do not promote derivative/community data without provenance review.

6. ACQUISITION AND NORMALIZATION IMPLEMENTATION

General downloader (scripts/download_datasets.py):

- Queries OpenCity CKAN metadata and downloads resources separately.
- Downloads Open-Meteo weather history for central Delhi (2017-2025) and CAMS history (2023-2025).
- Downloads CPCB reference PDFs.
- Uses temporary .part files, retries failed transfers, and records size/hash/provenance manifests.

CPCB/OpenCity normalization (scripts/normalize_cpcb.py):

- Renames source pollutant and weather column labels to a stable schema.
- Parses timestamps as UTC and retains an IST timestamp view.
- Converts numeric fields, maps known provider sentinel values (e.g. 999, 998, 985, 1999, -9999, -999) to missing.
- Resamples station data hourly by mean and retains sample counts.
- Requires at least two within-hour samples for PM2.5/PM10 to retain that hour's pollutant value.
- Writes per-station Parquet plus a source manifest.

Legacy AQI normalization:

- Has a dedicated script for the 2017-2023 AQI-only archive.
- Keeps this archive separate so AQI values are not misused as concentration targets.

OpenAQ (download_openaq.py / normalize_openaq.py):

- Uses OpenAQ API v3 with a 64-character key from OPENAQ_API_KEY.
- Retrieves Delhi/NCR locations and selected pollutant sensor IDs, catalog metadata, latest values, and resumable hourly-average batches.
- Paces requests and retries transient/429/server responses.
- Stores raw responses, local/source timestamps, sensor/location/parameter IDs, units, coverage metadata, and per-batch manifests.
- Normalization collapses duplicate sensor/time records, writes Parquet atomically, and stores row counts, range, and output hash.

FIRMS (download_firms.py / normalize_firms.py):

- Requires FIRMS_MAP_KEY or NASA_FIRMS_MAP_KEY outside the repo.
- Requests MODIS and VIIRS products in windows of at most five days over a Delhi + North India bounding box.
- Uses resumable per-window status records, retries, and SHA-256 hashes.
- Converts fire event times/coordinates/FRP/confidence, deduplicates events, and calculates daily totals, high-confidence counts, and DAPI.

AirDelhi and Kaggle/Hugging Face acquisition:

- AirDelhi normalizer makes an hourly gridded artifact and manifest separately.
- Kaggle download uses external CLI authentication and keeps credentials outside the repository.
- Public mirrors/datasets have their own manifests/roles; they are not joined into the core CPCB supervised panel by default.

Secrets:

DATA_GOV_API_KEY, OPENAQ_API_KEY, FIRMS_MAP_KEY, and WAQI_API_KEY are examples of credentials expected in the process environment or a local ignored file. Source scripts intentionally avoid committing them or writing them to manifests.

7. QUALITY CONTROL AND DATASET CONSTRUCTION

Panel audit (scripts/analyze_cpcb_panel.py):

- Reads normalized station Parquet files and calculates station-specific rows/ranges/coverage and network availability.
- Produces station_quality.csv, station_registry.csv, network_hourly_coverage.csv, and panel_quality.json.
- Uses conservative unique-name coordinate matches against a live CPCB snapshot. It does not fabricate coordinates for ambiguous/unmatched.

Reported audit results:

- 39 stations; 684,216 hourly rows.
- Coverage: 2024-01-01 00:00 UTC through 2025-12-31 23:00 UTC.
- Mean network hourly PM2.5 availability: 93.20%.
- No station below 60% structural hourly coverage.
- 38/39 station names matched to live CPCB coordinates; site_106 (IGI Airport) unmatched.

Primary feature dataset (scripts/build_multistation_dataset.py):

- Reads station-wise normalized files and reconciles them into a timestamp/station panel.
- Maintains missing values instead of inventing observations.
- Adds target columns by shifting each station's PM2.5 series forward by fixed horizons.
- Adds fixed nearest-station measurements at each issue timestamp.
- Writes Parquet plus manifest describing point-in-time policy, row counts, timestamps, feature names, neighbour map, and output hash.

Unified data (scripts/build_unified_dataset.py):

- Starts from the multi-station CPCB panel.
- As-of aligns weather onto issue times using only timestamps not later than the issue hour.
- Converts wind speed from km/h to m/s and direction to vector components u/v to encode circular direction sensibly.

- Adds weather current features and lagged weather features.
- Adds CAMS PM2.5/PM10 and gas features, with an explicit required six-hour publication lag for CAMS and additional lag/rolling features.
- Joins fire data only from complete prior days; rolling three- and seven-day fire aggregates are available.
- Writes a source-role manifest documenting included, validation-only, and excluded data sources.

8. FEATURE ENGINEERING AND LEAKAGE PROTECTION

For the historical station panel, `scripts/build_multistation_dataset.py` defines:

- PM2.5 current value.
- PM2.5 lags at 1, 3, 6, 12, 24, 48, and 72 hours.
- PM2.5 rolling mean and standard deviation over 3, 6, 12, 24, and 72 hours; rolling windows require a minimum fraction of samples.
- Current pollutant values and lags at 1, 3, 6, and 24 hours for PM10, NO2, SO2, CO, O3, and NH3.
- Current weather fields and lags at 1, 3, 6, and 24 hours from the station panel where available.
- Cyclic hour-of-day, day-of-week, and month sin/cos terms plus weekend flag.
- Five-nearest-station context: current mean, max, standard deviation, and number of neighbours reporting at that hour.
- Direct future PM2.5 target columns at +1, +3, +6, +12, and +24 hours.

The unified table then adds:

- Central weather current values and lags.
- CAMS pollutant products with availability/publish-delay rules.
- FIRMS counts, fire radiative power (FRP), confidence, regional totals, and rolling previous-day summaries.

Leakage controls present in code/design:

- Feature inputs are available at issue time t or earlier; targets are shifted to $t+h$.
- Station-wise shifts and joins avoid crossing station series.
- As-of joins use backward direction, never future weather values.
- CAMS features carry a conservative six-hour release lag.
- FIRMS features use completed prior days only.
- Train/validation/test partitions are chronological; no random shuffle for time splits.
- Direct horizon models avoid recursively feeding generated predictions into later forecasts.
- Multi-pollutant experiment purges issue rows whose target time would land in test.
- Sequence and GNN normalizers are fitted on the training interval only.

Remaining validation need: code-level point-in-time policies reduce obvious leakage, but an external audit of every source timestamp convention and every re-run manifest is still appropriate before claiming operational forecasting reliability.

9. MODEL PROGRAM: BASELINES, PROMOTED CANDIDATES, CHALLENGERS

9.1 Persistence baseline

At each issue time, persistence predicts the current PM2.5 value unchanged at the requested horizon. It is simple, hard to beat at very

9.2 Initial single-station HistGradientBoosting baseline

`scripts/build_forecast_dataset.py` and `scripts/train_baseline.py` establish an early one-station +1h pipeline for site_105.

HistGradientBoostingRegressor is trained on selected numeric fields with at least 80% nonmissingness. This is an initial baseline, separate from the final multi-station model.

9.3 Global multi-station XGBoost baseline

`scripts/train_multistation_baseline.py` pools station rows, encodes station identity and uses engineered station/time/neighbour features. It is trained separately at direct forecast horizons. Early run used a chronological split, with station-specific metrics retained so aggregate gain cannot hide station degradation.

9.4 Final unified-feature XGBoost models

`scripts/train_final_models.py` fits one XGBRegressor per horizon on all data before 2025-10-01 and evaluates the Oct–Dec test tail. Documented settings include squared-error regression objective, 350 trees, learning rate 0.05, depth 8, min-child weight 8, row subsample 0.9, column subsample 0.85, L2 regularization 2, histogram tree method, and fixed random seed 42. Predictions are clipped nonnegative at serving/evaluation boundaries. Each joblib bundle stores model, feature names, station encoding, horizon, feature set, point-in-time policy, and generation metadata. Station metrics are retained in result JSON.

9.5 Feature ablations and tree alternatives

`scripts/compare_model_strategies.py` tests CPCB core features, +weather, +weather+CAMS, and all sources, with XGBoost at 1/6/24 hour defaults. It also compares XGBoost with scikit-learn HistGradientBoosting on all sources. Training is deterministically thinned to every third row per station to limit CPU cost, while validation and test rows stay complete. A simple model-blend weight is selected only on validation MAE using a grid from 0.00 to 1.00; test is scored after selection.

9.6 Expanding-window backtests

`scripts/evaluate_backtests.py` evaluates quarterly windows from origins 2024-10-01, 2025-01-01, 2025-04-01, 2025-07-01, and 2025-10-01. Each fold expands its training history and scores the next three months. It reports overall and per-station scores. This adds seasonal robustness evidence beyond one split, although the available historical horizon remains short.

9.7 LSTM sequence challenger

`scripts/train_sequence_model.py` constructs contiguous hourly history windows, default comparison length 48 hours. Per-row features include PM2.5, neighbour mean/count, temperature, relative humidity, wind speed/direction, rain, and cyclic hour/weekday. Missingness

indicators are concatenated to values. Train-only medians, means, scales, and log1p target normalization are stored. The model is a 2-layer LSTM with 64 hidden units, dropout 0.1, an 8-dimensional station embedding, and a regression head. Optimization uses SmoothL1 loss, gradient clipping, Adam-family training as configured by the script, validation-based selection, and a fixed seed.

9.8 TCN sequence challenger

The temporal convolutional network shares the sequence data/cohort protocol. It projects inputs to 64 channels and uses causal-style residual convolution blocks with dilations 1, 2, 4, and 8, kernel size 3, GELU activations, normalization, and an 8-dimensional station embedding. It predicts the selected direct horizon from the final temporal representation.

9.9 Spatial GNN challenger

scripts/train_spatial_gnn.py makes coordinate-matched stations graph nodes and builds an undirected six-nearest-station adjacency with self-loops and row normalization. Node-level inputs include current PM2.5, 1h/3h PM2.5 lags, local temperature/humidity/wind/rain, and cyclic hour. A hidden feature transform plus neighbour message aggregation and residual/layer normalization feeds a regression head. Missing inputs use training-only medians; features are training-normalized; targets use standardized log1p. Early stopping selects on the July–September 2025 validation period. The evaluation also scores final XGBoost and persistence on identical station-hour keys.

9.10 Multipollutant AQI experiment

scripts/train_multipollutant_aqi.py trains direct horizon regressors for PM2.5, PM10, NO2, CO, and O3. PM2.5 reuses the final XGBoost artifact; other pollutant models are XGBoost. AQI is computed from the available predicted pollutant sub-indices with CPCB breakpoint logic. The diagnostic defaults to every third eligible training row per station, while full fitting can use all eligible pre-test history. It compares predicted subset AQI with a PM2.5-only proxy and persistence on matched cohorts.

9.11 WAQI live index models

scripts/train_waqi_live_index.py uses CPCB histories for Pusa/R.K. Puram, maps PM2.5 concentration to a legacy US-EPA-scale PM2.5 index proxy, and trains horizon-specific HistGradientBoostingRegressor candidates. History challenger features include current index, cyclic local time, station one-hot fields, lags at 1/3/6/12/24/48/72h, rolling index mean/std, and other pilot site's contemporaneous index. Optional weather features use the latest Open-Meteo data at or before issue time. Weather is promoted only when it beats validation and improves test MAE by at least 1%. The API uses the challenger after meaningful history exists, optional eligible weather model when fresh, and a single-reading incumbent during warm-up.

10. TRAINING PROTOCOLS AND MODEL SELECTION
Main historical split documented by model scripts:
- Training: issue time before 2025-07-01 UTC.
- Validation: 2025-07-01 through 2025-09-30 UTC.
- Test: issue time on/after 2025-10-01 UTC.

Final serving model procedure:

- Train the artifact on the complete pre-test history (train + validation, before 2025-10-01).
- Keep the October–December test tail for final reported scoring in that final-model run.
- Save separate artifacts and result files for each direct horizon.

Metrics:

- PM2.5 MAE in $\mu\text{g}/\text{m}^3$: typical absolute error.
- RMSE: more sensitive to large errors/spikes.
- R^2 : explanatory comparison within the test cohort.
- AQI proxy MAE/category accuracy where PM2.5-only mapping is used.
- Station-level scores and sample counts to surface heterogeneous behavior.
- Model strategy diagnostics also report validation/test and feature families.

Selection principles:

- Tune model families/blends on validation only.
- Hold test until final comparison when possible.
- Use matching station-hour cohorts for candidate comparisons.
- Keep negative challenger results and regressions in EXPERIMENTS.md.
- Do not interpret a lower aggregate MAE as universal station/season improvement.

Important qualification: the experiment ledger says the test tail was reused in subsequent candidate comparisons and the multi-pollutant follow-up, so not every later experiment is an untouched independent benchmark. This report preserves that caveat.

11. RESULTS AND WHAT THEY MEAN

11.1 Panel quality

39 stations; 684,216 hourly rows; Jan 2024–Dec 2025; 93.20% average network PM2.5 availability; no station below 60% structural coverage; one unmatched coordinate station (site_106).

11.2 One-hour multi-station XGBoost versus persistence (reported held-out test)

Metric	Persistence	XGBoost
MAE ($\mu\text{g}/\text{m}^3$)	22.692	18.527

RMSE 37.143 29.853

R² 0.891 0.929

Aggregate MAE improvement: 18.3%. However, site_113 worsened from 7.956 to 12.006 MAE, so the ledger did not treat this one result as sufficient promotion evidence.

11.3 One-hour rolling quarterly evaluation

Across five expanding-window quarterly test periods: average fold MAE was 16.383 for persistence and 14.466 for XGBoost, an 11.7% decrease. XGBoost won 176 of 195 station-fold slices and lost 19. The largest reported regression remained site_113 in a high-pollution quarter.

11.4 Six-hour sequence cohort (6,708 same-cohort rows)

Model	MAE	RMSE
Persistence	66.812	94.043
Global XGBoost	42.814	63.032

LSTM 45.195 69.235

TCN 45.197 68.380

XGBoost had about 5.3% lower MAE than the two sequence models. Thus sequence architectures were retained as reproducible research challengers, not selected serving models.

11.5 Spatial GNN, final XGBoost, and persistence

Horizon	Rows	Persistence MAE	GNN MAE	XGBoost MAE
1h	80,224	22.879	21.802	18.887
3h	79,664	48.141	37.979	35.339
6h	79,254	70.650	49.393	44.246
12h	78,780	85.446	58.956	50.901
24h	78,257	58.672	61.145	56.117

GNN is better than persistence at 1–12h but worse at 24h; XGBoost beats GNN at all listed horizons. This does not demonstrate a benefit

11.6 Six-hour feature family diagnostic

Feature set/model	Validation MAE	Test MAE
CPCB sensor/time/neighbours, XGBoost	11.834	47.930
+ weather, XGBoost	12.117	45.618
+ weather + CAMS, XGBoost	12.291	44.547
+ FIRMS (all sources), XGBoost	12.906	44.283
All sources, HistGradientBoosting	not reported	44.587
Validation-selected XGBoost/HGB blend	not reported	44.283

The lower summer validation errors and much higher polluted Oct-Dec test errors indicate substantial distribution/season differences. W

11.7 Six-hour five-pollutant AQI subset

On a matched 81,074-row cohort where PM2.5 exists:

- Five-pollutant AQI subset: MAE 47.10; category accuracy 62.10%.
- PM2.5-only proxy against observed composite AQI: MAE 51.68; category accuracy 57.77%.
- AQI persistence: MAE 75.97; category accuracy 47.18%.

This reports a 4.58-point MAE and 4.33 percentage-point category-accuracy improvement over the PM2.5-only proxy. It remains a five-poll

11.8 WAQI live pilot test split

Tracked evaluation.json uses training up to 2025-07-01, validation Jul–Sep 2025, test beginning Oct 2025. Test MAE (WAQI index points):

Horizon	History/spatial	Single-reading incumbent	Weather candidate
1h	19.334	20.209	19.194
3h	34.590	37.296	33.117
6h	42.583	47.630	39.635
12h	48.179	51.489	44.774
24h	50.691	52.407	50.024

The weather promotion gate requires both validation improvement and at least 1% test MAE improvement. Recorded eligible horizons: 12h a

12. AQI COMPUTATION AND FORECAST SEMANTICS

src/delhi_aircast/aqi.py defines concentration breakpoint tables and AQI categories from Good to Severe. It rounds pollutant concentration to integer precision, interpolates a pollutant sub-index within its breakpoint range, and returns the maximum valid pollutant sub-index for a composite result. It requires at least three valid pollutant sub-indices and at least PM2.5 or PM10. Missing/invalid data is not silently substituted.

Two result modes matter:

- ``ok``: enough pollutants are present for the configured CPCB composite calculation.
- ``insufficient_for_overall` / `pm25_derived_proxy``: an overall AQI cannot be calculated from the inputs, though the PM2.5 sub-index is present.

At non-six-hour horizons, the web interface uses a PM2.5-only proxy unless multipollutant artifacts are available for that requested horizon. Project README says its current multi-pollutant UI artifacts are focused at six hours. Labels in frontend distinguish the five-pollutant subset from a PM2.5 proxy.

13. API, DASHBOARDS, AND USER-FACING BEHAVIOR

FastAPI service (``src/delhi_aircast/api.py``):

- ``GET /health``: reports whether AQI engine, station registry, datasets, offline horizons, live pilot weights, and WAQI key are configured.
- ``POST /aqi``: validates only known pollutant fields; returns value, category, status, and pollutant sub-indices.
- ``GET /stations``: returns station metadata and latest offline feature rows when present.
- ``GET /stations/forecast?horizon=...``: scores station snapshots with a selected horizon model.
- ``GET /forecast/{station_id}?horizon=...``: station-specific historical model result.
- ``GET /forecast/{station_id}/path``: evaluates available direct model horizons.
- ``GET /stations/{station_id}/history?hours=...``: returns recent offline PM2.5 series.
- ``GET /live/stations/forecast?horizon=...&refresh=...``: requests WAQI for two stations, checks maximum two-hour age, builds history/snapshots.

Model loading:

- Offline resolver prefers ``data/runs/final_xgboost/model_{h}.joblib``, then the multi-station baseline.
- Historical features come from compact ``delhi_serving_features.parquet`` if present or the unified research table for local fallback.
- Six-hour route loads PM10/N02/CO/O3 models if all four extra artifacts are available; it combines those with PM2.5 and otherwise returns PM2.5 only.
- Offline API labels outputs ``historical_artifact`` and ``offline_historical_panel``.

Next.js dashboard (``aircast-web``):

- React client loads historical station forecast, station history, and live pilot endpoints using SWR.
- Allows horizon selection and station selection.
- Provides station map with clickable/keyboard-selectable monitoring markers, color-coded forecast AQI category, recent PM2.5 chart, compact history, and live pilot panel.
- Map explicitly notes markers are monitors, not a continuous surface.
- Separate live pilot panel shows Pusa/R.K. Puram WAQI index readings and predictions, feed age, warm-up/history/weather mode, and manual refresh.
- On an API/data-bundle error, historical network panel explains that it is absent from the lightweight deployment while the independent research bundle is available.
- Frontend uses responsive CSS and a Next.js rewrite from ``/backend/:path*`` to configured ``AIRCRAFT_API_URL``.

Streamlit dashboard (``dashboard.py``):

- Intended for the full local offline research bundle.
- Loads unified Parquet, maps station forecasts, provides station/time/horizon comparisons and metrics.
- Can compare compatible XGBoost, persistence, GNN, and 6-hour LSTM/TCN saved artifacts for a common issue row.
- Shows a simple mean of learned predictions only as experimental/unvalidated; XGBoost remains selected.
- Does not fetch live data.

14. PACKAGING, HOSTING, AND OPERATIONS

Serving bundle:

``scripts/build_serving_bundle.py`` assembles the latest offline feature row per station, coordinates, short histories, selected model artifacts, and optional six-hour pollutant artifacts. It intentionally omits training/raw datasets. README estimates about 48 MB locally; the portable export is described around 15 MB compressed / 44 MB expanded depending packaging. The bundle is historical at export time and does not refresh itself.

Docker:

- Python API image uses ``ghcr.io/astral-sh/uv:python3.13-bookworm-slim``.
- API image installs locked no-dev dependencies, source package, and tracked live pilot models.
- Docker Compose mounts only the serving data bundle read-only into ``/app/data`` and stores live history in a separate bind mount; WAQI key is in environment.
- Frontend image uses Node 22 Alpine, npm ci, Next standalone output, and API URL configuration.
- API has a healthcheck; web depends on healthy API.

Portable/local launch:

- Build ``local-serving`` or portable bundle from a complete research artifact set.
- Start uvicorn with ``DELHI_AIRCRAFT_DATA_ROOT`` pointed to bundle root.
- Install/run frontend with Node 22+ and set ``AIRCRAFT_API_URL``.
- README documents a Windows PowerShell portable launcher.

Hosted demo:

README lists a Vercel dashboard, Render API, Kaggle training data, and Hugging Face historical model repository. This report does not claim those endpoints are currently available or measure their uptime. Free hosting may sleep and ephemeral disks may lose collected live history, as README warns.

Operational controls in code:

- WAQI observations over two hours old are stale.
- WAQI response and PM2.5 value are validated.
- Upstream requests have timeouts and user-agent headers.
- WAQI responses cached five minutes; weather cached fifteen minutes.
- Weather is selected only as of or before sensor issue time and rejected if over two hours old.
- Live history appends are deduplicated by station/source observation timestamp; writes are best effort so logging failure does not stop.

15. REPOSITORY MAP AND KEY SCRIPTS

Top-level docs: README.md, problem.md, DATA_SOURCES.md, KAGGLE_DATASETS.md, MODEL_STRATEGY.md, EXPERIMENTS.md.

Acquisition:

- download_datasets.py – OpenCity/Open-Meteo/CAMS/reference data.
- download_cpcb_live.py – CPCB/data.gov.in live snapshot.
- download_openaq.py – OpenAQ catalog/latest/hourly API batches.
- download_firms.py – NASA FIRMS event windows.
- download_kaggle.py – optional Kaggle comparison datasets.

Normalization/audit/build:

- normalize_cpcb.py – hourly primary panel.
- normalize_cpcb_legacy_aqi.py – historical AQI-only archive.
- normalize_openaq.py, normalize_firms.py, normalize_airedelhi.py – source-specific normalized outputs.
- analyze_cpcb_panel.py – coverage/coordinates/network reports.
- build_forecast_dataset.py – early one-site baseline features.
- build_multistation_dataset.py – station-hour core table.
- build_unified_dataset.py – weather/CAMS/FIRMS additions.
- build_serving_bundle.py – compact inference package.

Model/evaluation:

- train_baseline.py – early HistGradientBoosting station baseline.
- train_multistation_baseline.py – pooled station XGBoost challenger.
- train_final_models.py – final per-horizon unified XGBoost.
- compare_model_strategies.py – feature-family, tree, and blend diagnostic.
- evaluate_backtests.py – expanding quarterly backtests.
- train_sequence_model.py / compare_sequence_cohort.py – LSTM/TCN experiments.
- train_spatial_gnn.py – station graph neural challenger.
- train_multipollutant_aqi.py – multi-pollutant six-hour AQI experiment.
- train_waqi_live_index.py – two-site live index model training.
- export_hf_historical_model.py – package historical weights/model card for Hugging Face.

Application:

- src/delhi_aircast/aqi.py – AQI calculations.
- src/delhi_aircast/api.py – API and live/offline inference.
- dashboard.py – Streamlit research UI.
- aircast-web/app/page.tsx and globals.css – Next.js dashboard.
- Dockerfile, docker-compose.yml, aircast-web/Dockerfile – container build/run.
- tests/ – AQI, API, normalization, dataset, model and evaluation unit coverage.

16. REPRODUCTION GUIDE

Prerequisites: Python 3.13+, uv, Node.js 22+ for web UI; API keys for gated sources. Start in project root.

Install/check environment:

- `uv sync --locked`
- `npm --prefix aircast-web ci`

Acquire and normalize the primary research panel:

- `uv run python scripts/download\_datasets.py --source all`
- `uv run python scripts/normalize\_cpcb.py`
- `uv run python scripts/analyze\_cpcb\_panel.py`

Build core and unified feature datasets:

- `uv run python scripts/build\_multistation\_dataset.py`
- `uv run python scripts/build\_unified\_dataset.py`

Run baseline/model diagnostics:

- `uv run python scripts/train\_multistation\_baseline.py --horizon 1`
- `uv run python scripts/compare\_model\_strategies.py --horizons 1 6 24 --n-jobs 4`
- `uv run python scripts/evaluate\_backtests.py --horizon 1 --model persistence`
- `uv run python scripts/evaluate\_backtests.py --horizon 1 --model xgboost --n-estimators 120 --n-jobs 6`
- `uv run python scripts/train\_sequence\_model.py --model lstm --horizon 6 --sequence-length 48 --stride 12 --epochs 3`
- `uv run python scripts/train\_sequence\_model.py --model tcn --horizon 6 --sequence-length 48 --stride 12 --epochs 3`
- `uv run python scripts/compare\_sequence\_cohort.py --horizon 1 --model tcn --horizon 6 --sequence-length 48 --stride 12`
- `uv run python scripts/train\_spatial\_gnn.py --horizon 6 --epochs 20 --lr 0.001 --patience 5`
- `uv run python scripts/train\_multipollutant\_aqi.py --horizon 6 --run-dir data/runs/multipollutant\_aqi\_final`
- `uv run python scripts/train\_final\_models.py --n-jobs 4`

Optional auxiliary sources:

- Set OPENAQ_API_KEY before `download\_openaq.py` batches.
- Set FIRMS_MAP_KEY before `download\_firms.py` date-range acquisition.
- Use `normalize\_openaq.py` / `normalize\_firms.py` after download.
- Acquire live CPCB snapshot with DATAGOV_API_KEY.

Serve local/offline data:

- ``uv run python scripts/build_serving_bundle.py --output .local-serving``
- Set ``DELHI_AIRCAST_DATA_ROOT`` to bundle root.
- ``uv run uvicorn delhi_aircast.api:app --reload --host 127.0.0.1 --port 8000``
- In another terminal: ``cd aircast-web``, set `AIRCAST_API_URL` to API, then ``npm run dev``.
- For a live pilot, configure `WAQI_API_KEY` in the API environment.
- Streamlit research UI: ``uv run streamlit run dashboard.py`` with full local feature/model artifacts.

CI instructions in README also list ``uv run pytest -q``, frontend typecheck, lint, build, npm audit, and a browser smoke script. The current report-generation task did not execute these project tests or deployments.

17. STRENGTHS, LIMITATIONS, RISKS, AND NEXT WORK

Strengths:

- Broad end-to-end implementation, not just a trained estimator: provenance, acquisition, normalization, audits, features, evaluation,
- Source roles are explicit and distinct datasets are not indiscriminately merged.
- Point-in-time joins, target shifts, chronological splits, purging, and train-only neural normalization address major leakage risks.
- Baselines, station-level comparisons, rolling backtests, and negative model results are recorded.
- AQI sufficiency and PM2.5 proxy status are separated in implementation and UI.
- Live readings are separate from offline model snapshots; user interfaces expose age/source/quality modes.
- APIs have unit tests and CI config; manifests record checksums and schema/provenance.

Limitations:

- Research data and final historical XGBoost models are absent in this Git checkout; the full historical report cannot be reproduced from this checkout.
- Main evaluation is based on a relatively short 2024-2025 window. The test tail is reused by later diagnostics; forward-period independence is not guaranteed.
- Aggregate gains conceal station and event regressions; site_113 is a concrete example.
- Accuracy degrades/varies with horizon and pollution regime. Summer validation is much easier than the polluted test tail.
- The GNN and LSTM/TCN challengers did not beat XGBoost on the measured cohorts, despite added complexity.
- Proposed prediction intervals/uncertainty outputs are not evidenced as implemented in current serving results.
- The five-pollutant AQI subset omits SO2 and NH3 and must not be called a complete pollutant forecast.
- Live WAQI scope is two stations, depends on key/upstream freshness, and requires persistent collection for useful lag features. Hosted data is not continuous.
- A station map is not continuous neighbourhood interpolation; monitor density and coordinates limit spatial claims.
- The API/web deployment URLs were not checked here. Present uptime, current model bundles, secrets, and persisted history remain operational.

Recommended work:

1. Rebuild from source manifests, verify downloaded files/hashes, and publish a versioned data-bundle recipe.
2. Create a genuinely untouched future test period and repeat seasonal rolling-origin evaluation.
3. Report station, event, horizon, season, missingness, and AQI-category slices with sample counts and underprediction rates.
4. Address station regressions through station-aware calibration or robust feature/quality review and re-test before promotion.
5. Implement calibrated prediction intervals and check coverage by station, horizon, and pollution regime.
6. Extend pollutant model coverage or keep incomplete AQI outputs explicitly labelled as subsets/proxies.
7. Prospectively evaluate WAQI live forecasts, archive history persistently, and monitor feed staleness/availability.
8. Add end-to-end serving checks for missing bundle, stale source, model-schema mismatch, and fallback mode.
9. Keep presentation language precise: concentration vs index, CPCB composite vs proxy, historical vs live, station vs neighbourhood.

18. FINAL ASSESSMENT

Delhi AirCast is a substantial, carefully structured research-and-demonstration project. Its core engineering work spans public data acquisition, source-aware normalization, station network auditing, point-in-time feature tables, multiple candidate models, AQI computation, APIs, dashboards, and portable/container deployment. The strongest current empirical direction is the global gradient-boosted tree model, which beats persistence overall in the cited tests and outperforms the evaluated neural and graph challengers on matched cohorts. The most important scientific/product caveats are limited historical evaluation, heterogeneous station performance, reused test-tail diagnostics, missing full data/model artifacts in this checkout, incomplete multi-pollutant coverage, and the two-site/US-EPA-scale nature of the live pilot.

A credible project presentation should explain not only “AI forecasts air quality,” but the complete chain: which observations are trusted, how they are normalized and time-aligned, how future leakage is avoided, why candidate models were retained or rejected, how error changes by station/horizon, what AQI labels mean, and which displayed values are live versus historical. Those distinctions are included in this report so it can support technical review as well as a high-level project explanation.

PRIMARY REFERENCES INSPECTED

README.md; problem.md; DATA_SOURCES.md; KAGGLE_DATASETS.md; MODEL_STRATEGY.md; EXPERIMENTS.md; pyproject.toml; Dockerfile; docker-compose.yml; .github/workflows/ci.yml; scripts/build_multistation_dataset.py; scripts/build_unified_dataset.py; scripts/train_baseline.py; scripts/train_multistation_baseline.py; scripts/train_final_models.py; scripts/compare_model_strategies.py; scripts/evaluate_backtests.py; scripts/train_sequence_model.py; scripts/train_spatial_gnn.py; scripts/train_multipollutant_aqi.py; scripts/train_waqi_live_index.py; scripts/build_serving_bundle.py; scripts/download_datasets.py; scripts/download_openaq.py; scripts/download_firms.py; scripts/normalize_cpcb.py; scripts/normalize_openaq.py; scripts/normalize_firms.py; scripts/analyze_cpcb_panel.py; src/delhi_aircast/aqi.py; src/delhi_aircast/api.py; dashboard.py; aircast-web/app/page.tsx; aircast-web/app/layout.tsx; aircast-web/next.config.js; aircast-web/Dockerfile; models/waqi_live_index/evaluation.json; existing project report and slide deck.

END OF REPORT